Project's Low-Level Design

on

Dockerized Marketing Planner Agent

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Akhilesh Tiwari	EN22CS301085
2	Aditi Gaikwad	EN22CS301040
3	Alfez Khan	EN22CS301102
4	Akshat Sakalye	EN22CS301093
5	Aanchal Chaturvedi	EN22CS301014
6	Aditya Patidar	EN22CS301060

Group Name: Group 06D1

Project Number: DO-06

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction
    a. Scope of the document
    b. Intended Audience
    c. System overview
2. System Design
    a. Application Design
    b. Process Flow
    c. Information Flow
    d. Components Design
    e. Key Design Considerations
    f. API Catalogue
3. References

1 Introduction

1.1 Scope of the document

This document describes the low-level design of the Dockerized Marketing Planner Agent. It details the containerization strategy, multi-stage Docker architecture, internal APIs, and the deployment workflow on AWS EC2, laying the groundwork for future Kubernetes (K8s) and Jenkins integrations.

1.2 Intended Audience

Developers, DevOps Engineers, Testers, and Project Evaluators/Faculty.

1.3 System Overview

The system is an Agentic AI SaaS platform that decomposes high-level marketing goals into actionable schedules using LLMs (Groq/OpenRouter). The application is fully containerized using Docker to ensure consistent environments across local development and AWS production servers.

The containerized architecture consists of:

- **Frontend Service:** React SPA (Vite) built and served via Nginx.
- **Backend Service:** Python (FastAPI) handling business logic, CrewAI orchestration, and ChromaDB vector search.
- **Database Service:** Relational MySQL container for persistent storage.

The system utilizes **Docker Compose** for multi-container orchestration and networking, setting the foundation for future CI/CD automation and infrastructure-as-code (Terraform) deployments.

2 System Design

2.1 Application Design

The system transitions from a local monolithic execution into a distributed, containerized microservices architecture using Docker.

Layers & Containers:

1. **Frontend Service Container (React/Vite):**
    a. Serves the static SPA assets.
    b. Handles UI rendering and client-side routing.
2. **API/Backend Service Container (FastAPI):**
    a. Hosts the core Python logic and CrewAI orchestration.
    b. Exposes REST endpoints.
3. **Database Service Container (MySQL):**
    a. Manages persistent multi-tenant data storage.
    b. Isolated within a private Docker network.

Docker Compose Orchestration: A single docker-compose.yml file is used to orchestrate the lifecycle, networking, and environment variables of all three containers, simulating a production-ready cluster locally.

2.2 Process Flow (Dockerized Execution)

1. **Initialization:** The DevOps engineer executes docker compose up -d.
2. **Database Boot:** The mysql-db container starts first, initializing the schemas.
3. **Backend Boot:** The backend-api container starts, waiting for the database to be healthy. It installs dependencies and starts the Uvicorn server on port 8000.
4. **Frontend Boot:** The frontend-ui container starts, serving the React application on port 5173.
5. **Networking:** The frontend container routes API requests internally to the backend container using Docker's internal DNS resolution (e.g., http://backend:8000/api/v1/...).

2.3 Information Flow

- **Client to Frontend:** User interacts with the UI via port 5173.
- **Frontend to Backend:** HTTP/JSON payloads are transmitted over the Docker bridge network to port 8000.
- **Backend to Database:** SQLAlchemy ORM communicates securely over the internal network to the MySQL container on port 3306.

- **Backend to External LLMs:** The FastAPI container reaches out to the public internet to communicate with the Groq/OpenRouter APIs securely.

2.4 Component Design (Docker Architecture)

1. **The Backend Dockerfile (Multi-Stage Build)**
    - *Base Image:* python:3.10-slim (Minimizes attack surface and image size).
    - *Dependencies:* Installs CrewAI, FastAPI, SQLAlchemy, and ChromaDB via requirements.txt.
    - *Execution:* Uses uvicorn as the entry point.
2. **The Frontend Dockerfile**
    - *Build Stage:* Uses node:18-alpine to build the Vite production assets.
    - *Serve Stage:* Uses a lightweight web server (e.g., nginx:alpine or serve) to host the static files, drastically reducing the final image size.
3. **The Database Setup**
    - Uses the official mysql:8.0 image.
    - Implements Docker Volumes (mysql_data:/var/lib/mysql) to ensure data persistence even if the container is destroyed.

2.5 Key Design Considerations (DevOps & AWS Deployment)

1. **Dynamic Secrets Management (Solving the API Exhaustion Problem):**
    - *Design:* API keys (Groq/OpenRouter) are **never** baked into the Docker image. They are injected at runtime via an external .env file mounted to the container.
    - *Benefit:* If an API key is exhausted during the presentation, the engineer can simply edit the .env file on the AWS EC2 instance and run docker compose restart backend. The new keys are immediately active without rebuilding the image.
2. **Portability & AWS Deployment:**
    - The entire stack is encapsulated in standard Docker images. Deployment to the AWS free-tier EC2 instance only requires installing Docker, cloning the docker-compose.yml, and executing a single command.

3. **Future-Proofing for Kubernetes (K8s):**
   - By strictly separating the application into distinct containers (Frontend, Backend, DB) and externalizing state (Volumes/DBs) and configuration (.env), the architecture is perfectly aligned with 12-Factor App methodology. This makes the future migration from docker-compose to Kubernetes Manifests (Deployments, Services, Ingress) seamless.
4. **Image Optimization:**
   - Using -alpine and -slim base images reduces build times, minimizes storage costs on Docker Hub, and speeds up EC2 deployment times.

2.6 API Catalogue

The system is decoupled, communicating entirely over RESTful HTTP/JSON endpoints. All endpoints (except registration/login) require a valid Bearer JWT in the Authorization header.

Endpoint	Method	Payload / Request	Response / Purpose
/api/v1/auth/register	POST	{email, password, orgName}	Creates Organization & User. Returns JWT.
/api/v1/auth/login	POST	OAuth2PasswordRequestForm	Authenticates user. Returns JWT (access_token).
/api/v1/agent/plan	POST	{goal: "Analyze Q3 Ads"}	Triggers multi-agent Crew. Returns auto-saved Plan ID and HTML.

/api/v1/documents/upload	POST	multipart/form-data (PDF)	Vectorizes PDF and stores in ChromaDB with org_id metadata.
/api/v1/plans	GET	*None*	Returns array of PlanResponse objects for the user's tenant.
/api/v1/plans/{plan_id}	PUT	{title: "New Title"}	Renames a specific plan. Returns updated PlanResponse.
/api/v1/plans/{plan_id}	DELETE	*None*	Deletes a plan. Returns 204 No Content.

3 References

1. Docker Official Documentation (Multi-stage builds, Networking).
2. FastAPI in Containers Deployment Guide.
3. CrewAI Framework Architecture.